COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	August 04, 2026	**DATE SUBMITTED:**	August 06, 2026
NAME:	Garbosa, Isaiah Benedict R.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 5.1 Queues

1. PROCEDURE OUTPUT

ILO A

Code

```
#include <iostream>
#include <queue>

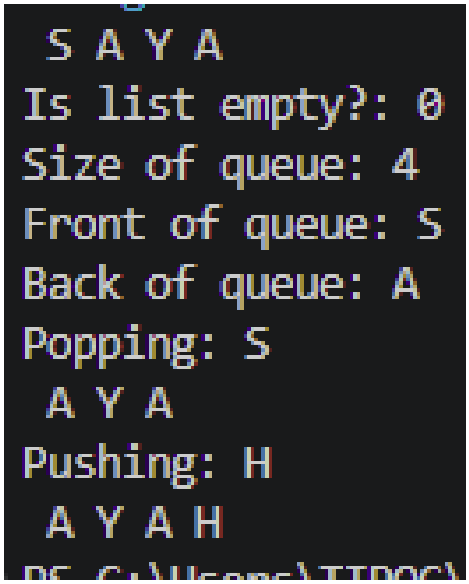
void display(std::queue<char> copyQ);


int main(){
    //create an object
    std::queue<char> myQ;

    myQ.push('S');
    myQ.push('A');
    myQ.push('Y');
    myQ.push('A');
    display(myQ);
    //try all member function
    //empty
    std::cout<<"Is list empty?: "<<myQ.empty()<<std::endl;
    //size
    std::cout<<"Size of queue: "<<myQ.size()<<std::endl;
    //front
    std::cout<<"Front of queue: "<<myQ.front()<<std::endl;
    //back
    std::cout<<"Back of queue: "<<myQ.back()<<std::endl;
```

```
25      //pop
26      std::cout<<"Popping: S"<<std::endl;
27      myQ.pop();
28      //display
29      display(myQ);
30      //push
31      myQ.push('J');
32      //display
33      display(myQ);
34
35      return 0;
36  }
37  void display(std::queue<char> copyQ){
38      //creates a copy of the queue that was passed
39      std::queue<char> c = copyQ;
40      while(!c.empty()){
41          std::cout<<" "<<c.front();
42          c.pop();
43      }
44      std::cout<<"\n";
45  }
46
```

Output



Observation

The code is all about the queues which are similar to linked lists. The queue library is imported and the different functions are used. The push adds a member to the queue, the empty checks if the list is empty, the front shows the front of the queue, the back shows the back of the queue, and the pop removes a member in the queue. I also created a display function that uses a while loop to display all the data in a queue.

ILO B.1

Main.cpp

```cpp
#include <iostream>
#include "queueLL.h"

int main (){

    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue('J', &front, &back);
    std::cout<<front->data<<" "<<back->data<<std::endl;
    enqueue('O', &front, &back);
    std::cout<<front->data<<" "<<back->data<<std::endl;
    enqueue('M', &front, &back);
    std::cout<<front->data<<" "<<back->data<<std::endl;

    dequeue(&front, &back);
    std::cout<<front->data<<" "<<back->data<<std::endl;
    dequeue(&front, &back);
    std::cout<<front->data<<" "<<back->data<<std::endl;

    enqueue('O', &front, &back);
    std::cout<<front->data<<" "<<back->data<<std::endl;
    enqueue('P', &front, &back);
    std::cout<<front->data<<" "<<back->data<<std::endl;

    checkFront(front);
    checkBack(back);
    display(front);
    return 0;
}
```

queueLL.h

```h
#ifndef QUEUELL_H
#define QUEUELL_H

template <typename T>
class qNode{
    public:
        T data;
        qNode<T>* next;
};

//creates the new node
template <typename T>
qNode<T>* new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

//enqueue operation
template<typename T>

void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    //create the new node
    qNode<T>* newNode = new_node(newData);
    //inserting into an empty queue
    if((*frontPtr) == nullptr && (*backPtr)==nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
    }

    //inseting an item into a non empty queue
    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
}
```

```
36
37  template<typename T>
38  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
39      //crete a temp node
40      qNode<T>* deleteNode = nullptr; //temp var
41      deleteNode = (*frontPtr);
42      //add a check if the queue is already empty
43      if((*frontPtr)== nullptr && (*backPtr)==nullptr){
44          std::cout<<"the queue is empty\n";
45      }
46      //check if the queue as only has 1 node
47      if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
48          (*frontPtr) = nullptr;
49          (*backPtr) = nullptr;
50          delete deleteNode;
51          return;
52      };
53      //delete the node
54      (*frontPtr) = (*frontPtr)->next;
55      deleteNode->next = nullptr;
56      delete deleteNode;
57  }
58
59  //check the front
60  template<typename T>
61  void checkFront(qNode<T>* front){
62      if(front == nullptr){
63          std::cout<<"List empty"<<std::endl;
64      }
65      std::cout<<"The front is: "<<front->data<<std::endl;
66  }
67
68  //check the back
69  template<typename T>
70  void checkBack(qNode<T>* back){
71      if(back == nullptr){
72          std::cout<<"List empty"<<std::endl;
73      }
74      std::cout<<"The back is: "<<back->data<<std::endl;
75  }
```

```
76
77  //display all
78  template<typename T>
79  void display(qNode<T>* front){
80      if(front == nullptr){
81          std::cout<<"List empty"<<std::endl;
82      }
83      qNode<T>* temp = front;
84      std::cout<<"Displaying: ";
85      while(front != nullptr){
86          std::cout<<front->data<<" ";
87          front = front->next;
88      }
89  }
90  #endif
```

Output

```
J J
J O
J M
O M
M M
M O
M P
The front is: M
The back is: P
Displaying: M O P
```

Observation
The code uses a linked list to create a queue. It creates a node, then uses that node to enqueue, which creates a new node, sets the front and back to the new node, and sets the next of the back node to the new node and the back node is set to the new node. The dequeue functions essentially the same back first setting a temporary node for the ptr and sets both front and back to null, then the front is set to next to null then deleting the node. The check front and check back essentially works the same by printing the data of either the front or back. Then, the display all functions by using a while loop that prints the data of the front, then, the front is set to the front->next and goes on until the front is a null.

ILO B.2

Main.cpp

```cpp
#include <iostream>
#include "queueArr.h"

int main(){

    QueueArray<char> queue(5);

    //Enqueue
    queue.Enqueue('S');
    queue.Enqueue('A');
    queue.Enqueue('Y');
    queue.Enqueue('A');

    std::cout<<"Front: "<<queue.Front()<<std::endl;
    std::cout<<"Back: "<<queue.Back()<<std::endl;
    std::cout<<"Size: "<<queue.Size()<<std::endl;

    //Dequeue
    queue.Dequeue();

    std::cout<<"\nAfter Dequeue"<<std::endl;
    std::cout<<"Front: "<<queue.Front()<<std::endl;
    std::cout<<"Back: "<<queue.Back()<<std::endl;
    std::cout<<"Size: "<<queue.Size()<<std::endl;

    queue.Clear();

    std::cout<<"\nAfter Clear"<<std::endl;
    std::cout<<"Front: "<<queue.Front();
    std::cout<<"Back: "<<queue.Back();
    std::cout<<"Size: "<<queue.Size()<<std::endl;



    return 0;
}
```

queueArr.h

```
#ifndef QUEUEARR_H
#define QUEUEARR_H

template <typename T>
class QueueArray{
private:
    T* q_array;
    int q_capacity;
    int q_size;
    int q_front;
    int q_back;
public:
    QueueArray(int capacity = 100){
        q_capacity = capacity;
        q_array = new T[q_capacity];
        q_size = 0;
        q_front = 0;
        q_back = -1;
    }

    QueueArray(const QueueArray<T> &other){
        q_capacity = other.q_capacity;
        q_size = other.q_size;
        q_front = other.q_front;
        q_back = other.q_back;
        q_array = new T[q_capacity];
        for(int i = 0; i < q_capacity; i++){
           q_array[i] = other.q_array[i];
        }
    }

    QueueArray<T>& operator=(const QueueArray<T> &other){
        if(this != &other){
            delete[] q_array;
            q_capacity = other.q_capacity;
            q_size = other.q_size;
            q_front = other.q_front;
            q_back = other.q_back;
            q_array = new T[q_capacity];
            for(int i = 0; i < q_capacity; i++){
                q_array[i] = other.q_array[i];
            }
        }
        return *this;
    }
```

```
47      ~QueueArray(){
48          delete[] q_array;
49      }
50
51      bool Empty(){
52          return q_size == 0;
53      }
54
55      int Size(){
56          return q_size;
57      }
58
59      void Clear(){
60          q_size = 0;
61          q_front = 0;
62          q_back = -1;
63      }
64
65      T Front(){
66          if(Empty()){
67              std::cout<<"Queue is Empety"<<std::endl;
68              return T();
69          }
70          return q_array[q_front];
71      }
72
73      T Back(){
74          if(Empty()){
75              std::cout<<"Queue is Empety"<<std::endl;
76              return T();
77          }
78          return q_array[q_back];
79      }
80
81      void Enqueue(T value){
82          if(q_size == q_capacity){
83              std::cout<<"Queue is Full"<<std::endl;
84              return;
85          }
86          q_back++;
87          if(q_back == q_capacity){
```

```
88              q_back = 0;
89          }
90          q_array[q_back] = value;
91          q_size++;
92      }
93
94      void Dequeue(){
95          if(Empty()){
96              std::cout<<"Queue is Empety"<<std::endl;
97              return;
98          }
99          q_front++;
100         if(q_front == q_capacity){
101             q_front = 0;
102         }
103         q_size--;
104
105         if(q_size == 0){
106             q_front = 0;
107             q_back = -1;
108         }
109     }
110
111 };
112
113 #endif
```

Output

```
Front: S
Back: A
Size: 4

After Dequeue
Front: A
Back: A
Size: 3

After Clear
Front: Queue is Empety
Back: Queue is Empety
Size: 0
```

Observation
The code works by creating a header file. The header file has a class called QueueArray with a private and public class. The private class holds the variables needed such as the array, capacity, size, front, and back. Then, the values are initialized using a constructor. A copy constructor, copy assignment operator, and destructor is also created. Next are the different operators. First is the bool empty which checks if the size is 0, if it is, then it means that it is empty. Next is the size which returns the size of the array, the clear operation resets the values of the size, front, and back. Then, the checking of the front and back operation. If the queue is empty, it returns a message, if not, it prints the front and back by using the array. The enqueue first checks if the size is full, if not, the back is incremented, it checks if the back is equal to the capacity after that, if not, the back index from the array's value would be set to the value we chose, the size is incremented. The dequeue does the same process, except this time, it adds to the front, making the next index the front, then it decrements the size.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C

main.cpp

```cpp
#include <iostream>
#include <string>
#include "printer.h"
#include "job.h"

int main(){

    Printer<Job> printer;

    int choice;
    int id;
    int pages;
    std::string user;

    while(choice != 4){
        std::cout << "\n===== Printer Queue Menu =====\n";
        std::cout << "1. Add Print Job\n";
        std::cout << "2. Process Next Job\n";
        std::cout << "3. Display Pending Jobs\n";
        std::cout << "4. Exit\n";
        std::cout << "Enter choice: ";
        std::cin >> choice;

        switch(choice){

        case 1:

            std::cout << "\nEnter Job ID: ";
            std::cin >> id;

            std::cout << "Enter User Name: ";
            std::cin >> user;

            std::cout << "Enter Number of Pages: ";
            std::cin >> pages;

            printer.addJob(Job(id, user, pages));
            std::cout << "\nJob added successfully!"<<std::endl;
            break;

        case 2:

            printer.processJob();
            break;
```

```
        case 3:

            printer.displayJobs();
            break;

        case 4:

            std::cout << "\nExiting"<<std::endl;
            break;

        default:

            std::cout << "\nInvalid choice."<<std::endl;
        }

    };

    return 0;
}
```

job.h

```
#ifndef JOB_H
#define JOB_H

#include <iostream>
#include <string>

class Job{
public:
    int id = 0;
    std::string user = "";
    int pages = 0;

    Job(int i, std::string u, int p){
        id = i;
        user = u;
        pages = p;
    }

    void display(){
        std::cout<<"Job ID: "<<id<<std::endl;
        std::cout<<"User: "<<user<<std::endl;
        std::cout<<"Pages: "<<pages<<std::endl;
    }
};

#endif
```

printer.h

```cpp
#ifndef PRINTER_H
#define PRINTER_H

#include "job.h"

template <typename T>
class Node{
public:
    T data;
    Node<T>* next;
    Node(T newData);
};

template <typename T>
Node<T>::Node(T newData) : data(newData), next(nullptr) {}

template <typename T>
class Printer{
private:
    Node<T>* front;
    Node<T>* back;
public:
    Printer();
    void addJob(T job);
    void processJob();
    void displayJobs();
    bool isEmpty();
};

template <typename T>
Printer<T>::Printer(){
    front = nullptr;
    back = nullptr;
}

template <typename T>
void Printer<T>::addJob(T job){
    Node<T>* newNode = new Node<T>(job);

    if(front == nullptr){
        front = back = newNode;
        return;
    }
    back->next = newNode;
```

```
45      back = newNode;
46  }
47
48  template <typename T>
49  void Printer<T>::processJob(){
50      if(front == nullptr){
51          std::cout<<"No pending jobs"<<std::endl;
52          return;
53      }
54      std::cout<<"\nProcessing job: "<<std::endl;
55      front->data.display();
56      Node<T>* temp = front;
57      front = front->next;
58
59      if(front == nullptr){
60          back = nullptr;
61      }
62      delete temp;
63  }
64
65  template <typename T>
66  void Printer<T>::displayJobs(){
67      if(front == nullptr){
68          std::cout<<"No pending jobs"<<std::endl;
69          return;
70      }
71      Node<T>* temp = front;
72
73      std::cout<<"\nPending Jobs: "<<std::endl;
74      while(temp != nullptr){
75          temp->data.display();
76          std::cout<<"--------------------"<<std::endl;
77          temp = temp->next;
78      }
79  }
80
81  template <typename T>
82  bool Printer<T>::isEmpty(){
83      return front == nullptr;
84  }
85
86  #endif
```

Output

Adding print job:

```
===== Printer Queue Menu =====
1. Add Print Job
2. Process Next Job
3. Display Pending Jobs
4. Exit
Enter choice: 1

Enter Job ID: 0920
Enter User Name: Ned
Enter Number of Pages: 10

Job added successfully!

===== Printer Queue Menu =====
1. Add Print Job
2. Process Next Job
3. Display Pending Jobs
4. Exit
Enter choice: 1

Enter Job ID: 0724
Enter User Name: MJ
Enter Number of Pages: 7

Job added successfully!

===== Printer Queue Menu =====
1. Add Print Job
2. Process Next Job
3. Display Pending Jobs
4. Exit
Enter choice: 1

Enter Job ID: 1202
Enter User Name: Peter
Enter Number of Pages: 8

Job added successfully!
```

Processing:

```
===== Printer Queue Menu =====
1. Add Print Job
2. Process Next Job
3. Display Pending Jobs
4. Exit
Enter choice: 2

Processing job:
Job ID: 920
User: Ned
Pages: 10
```

Display pending jobs:

```
===== Printer Queue Menu =====
1. Add Print Job
2. Process Next Job
3. Display Pending Jobs
4. Exit
Enter choice: 3

Pending Jobs:
Job ID: 724
User: MJ
Pages: 7
--------------------
Job ID: 1202
User: Peter
Pages: 8
--------------------
```

Processing All:

```
===== Printer Queue Menu =====
1. Add Print Job
2. Process Next Job
3. Display Pending Jobs
4. Exit
Enter choice: 2

Processing job:
Job ID: 724
User: MJ
Pages: 7

===== Printer Queue Menu =====
1. Add Print Job
2. Process Next Job
3. Display Pending Jobs
4. Exit
Enter choice: 2

Processing job:
Job ID: 1202
User: Peter
Pages: 8

===== Printer Queue Menu =====
1. Add Print Job
2. Process Next Job
3. Display Pending Jobs
4. Exit
Enter choice: 3
No pending jobs
```

Observation

The code has two header files, utilizing different classes. First is the job header file with the job class. This contains the information needed for the printing job. It contains the id, username, and pages of the document that will be printed. It also displays all the information. The printer header file uses a node class which contains variables needed for a node. Next is the printer class which has a private and public class. The private class creates the front and back node. The public class contains a constructor with function declarations. Then, a printer node is created from the constructor. The addJob function first creates a new node, it checks if the front is a null pointer, if it is the front and back is equal to the new Node. If not, the next of the back is set to the new node and the back is set to the new node. The processJob checks first if the front is a null, if not, it displays the processing job. It displays the data of the front first, then it creates a temporary node and sets it to the front. The front is then set to the next of the front. If the front is null, the back is null. Then, delete null. The display jobs

operation checks first if the front is null, if not, a temporary node is set to the front, then a while loop displays the information of the temporary node using the display function from the job class. Then, it sets the temp to the next of the temp. Finally the is empty which checks if the front is a null. The main uses a switch case and a while loop. The user can add a print job, process the current print job, and display all current print jobs. I chose the linked list because I believe that it is more flexible. The linked list queue stores data dynamically and I believe it's easier to access data.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.